

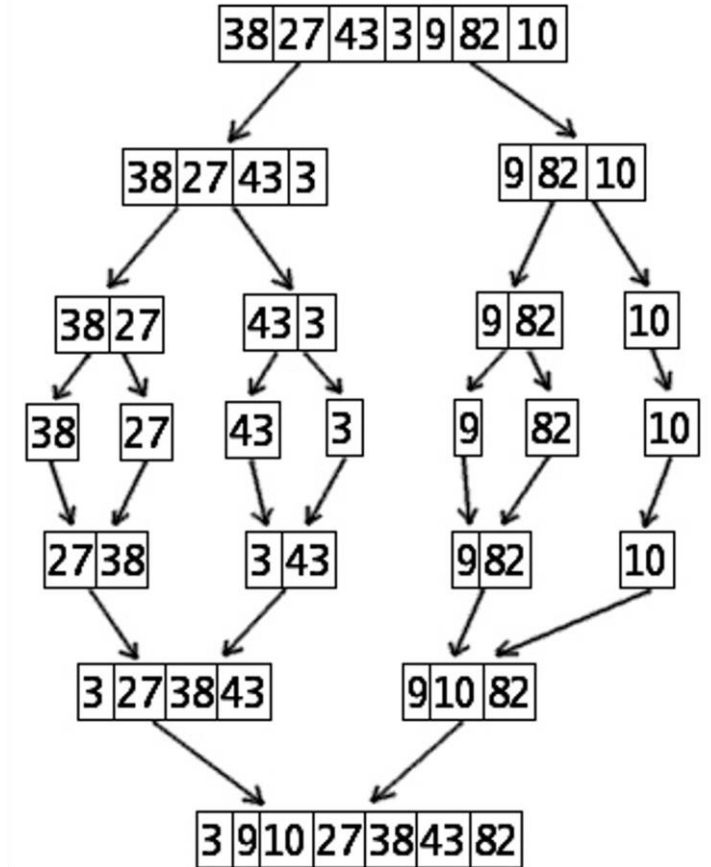
Advanced Sorting Algorithm



Merge Sort

Merge Sort

Sắp xếp hai dãy đã được sắp xếp thành một dãy số được sắp xếp



Merge Sort

- Tách dãy số ra làm hai phần
- Sắp xếp hai dãy số đó
- Trộn hai dãy số lại thành dãy số sắp xếp

38	27	43	3	9	82	10
----	----	----	---	---	----	----

38	27	43	3
----	----	----	---

9	82	10
---	----	----

3	27	38	43
---	----	----	----

9	10	82
---	----	----

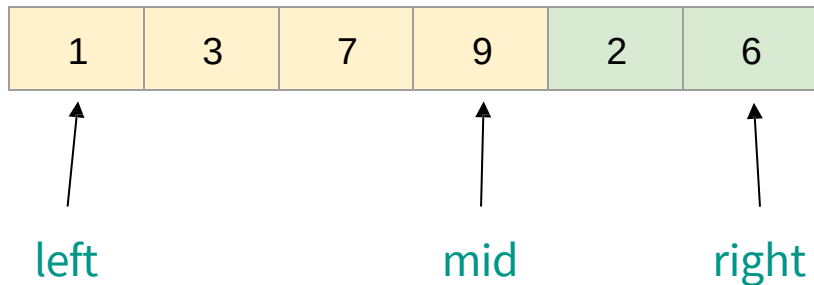
3	9	10	27	38	43	82
---	---	----	----	----	----	----

Megre

Cho hai dãy số đã được sắp xếp

- Dãy số A gồm n phần tử
- Dãy số B gồm m phần tử

Hãy trộn hai dãy số trên thành một dãy số C cũng được sắp xếp có $n + m$ phần tử



A	B		C
1, 3, 7, 9	2, 6	$1 < 2$	1
3, 7, 9	2, 6	$3 > 2$	1, 2
3, 7, 9	6	$3 < 6$	1, 2, 3
7, 9	6	$7 > 6$	1, 2, 3, 6
7, 9			1, 2, 3, 6
			1, 2, 3, 6, 7, 9

1	2	3	6	7	9
---	---	---	---	---	---

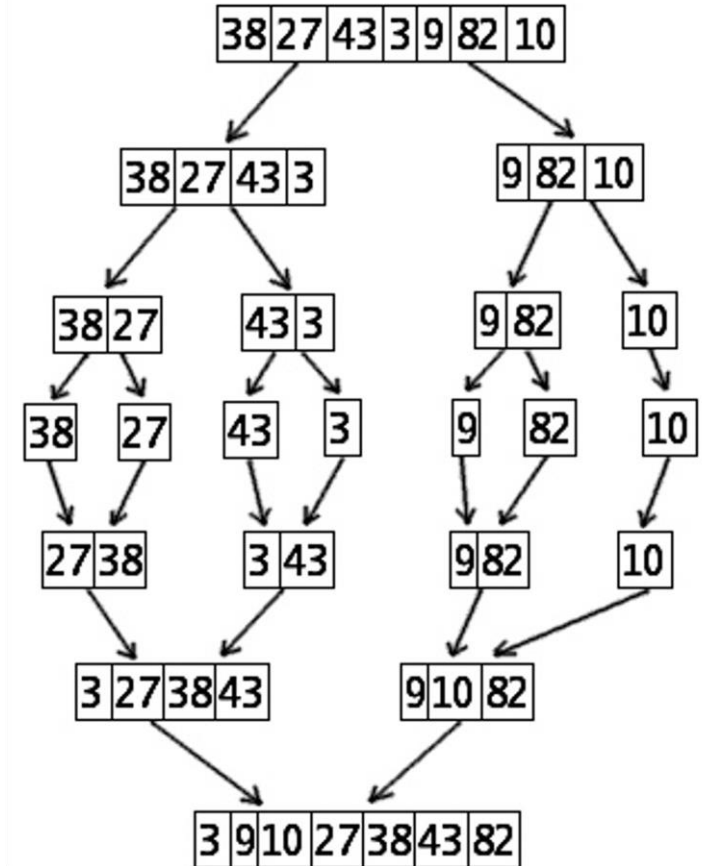
Megre

```
1 Megre( Arr, left, mid, right)
2     i = left;
3     j = mid + 1
4     v = left
5     while (i <= mid && j <= right)
6         if (Arr[i] < Arr[j])
7             C[v] = Arr[i]
8             i++
9         else
10            C[v] = Arr[j]
11            j++
12            v++
```

```
13     while (i <= mid)
14         C[v] = Arr[i]
15         i++
16         v++
17     while (j <= right)
18         C[v] = Arr[j]
19         j++
20         v++
21     return C
```

Merge Sort

```
1 MergeSort(Arr, left, right)
2     if (left < right)
3         mid = (left + right) / 2
4         MergeSort(Arr, left, mid)
5         MergeSort(Arr, mid + 1,
6 right)
7         Merge(Arr, left, mid,
```



Merge Sort

Độ phức tạp $O(n \log n)$

Giữ nguyên thứ tự của các phần tử bằng nhau

Độ phức tạp không liên quan đến INPUT

Có thể xử lý dữ liệu phân tán

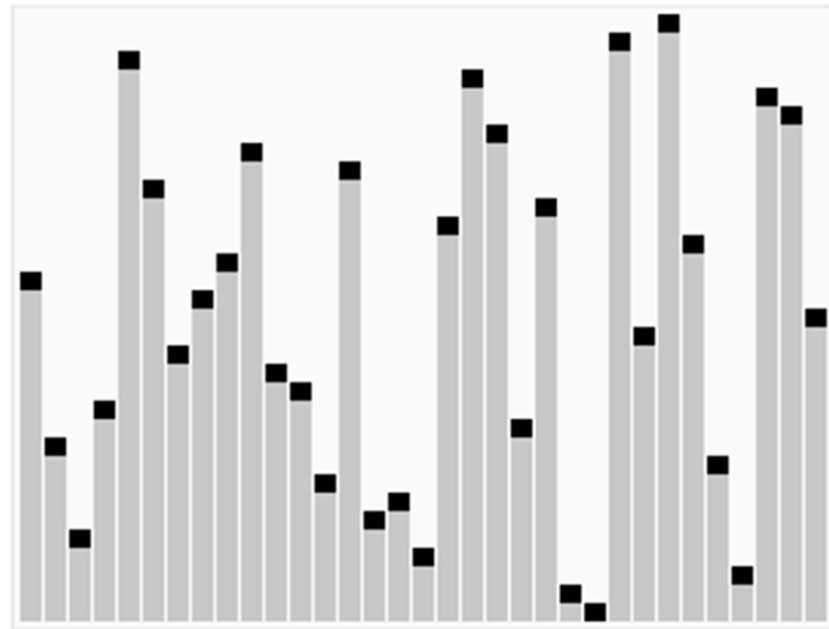
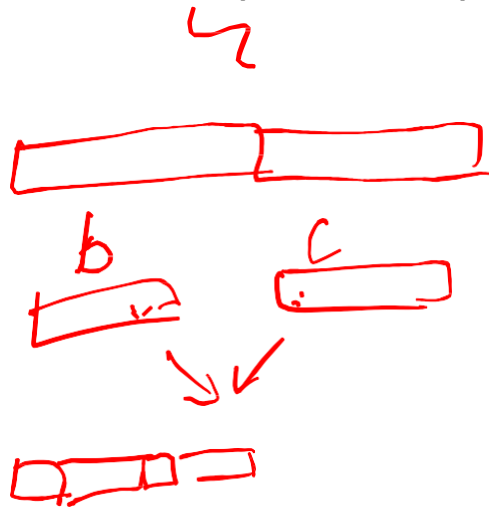
Cần thêm bộ nhớ $O(n)$

$O(n \log n)$ - đệ quy - chia đôi

Quick Sort

Quick Sort

Chia dãy số thành hai phần rồi sắp xếp hai phần đó



Quick Sort

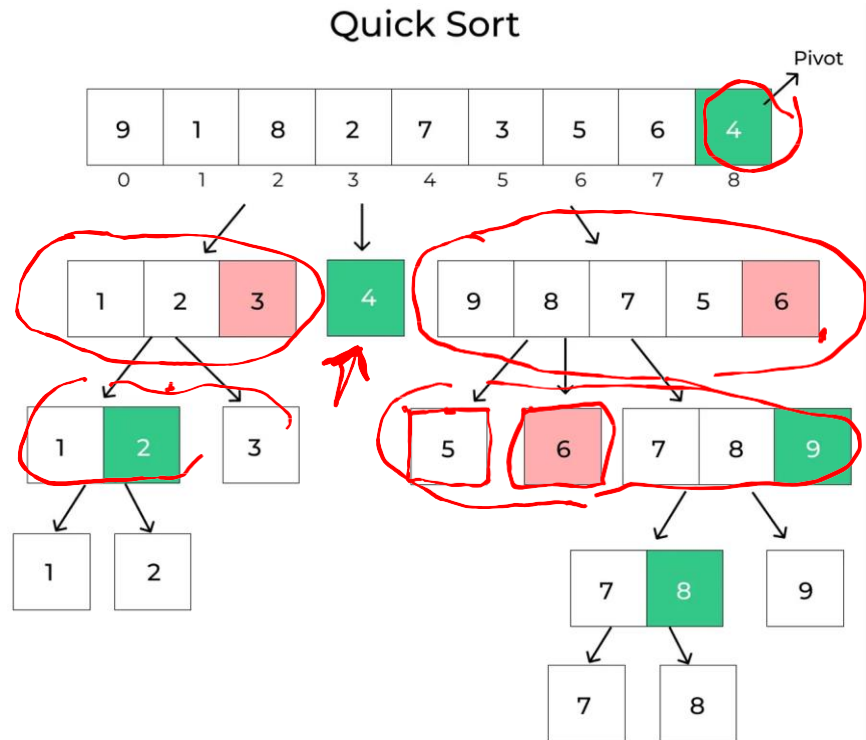
Chọn một giá trị khóa pivot

Chia dãy số thành hai phần:

Phần bên trái gồm các số nhỏ hơn hoặc bằng pivot

Phần bên phải gồm các số lớn hơn hoặc bằng pivot

Sắp xếp hai dãy số con

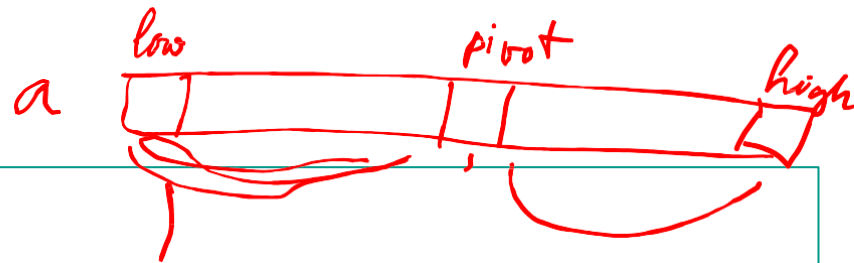


Quick Sort

```

1  quickSort(Arr, low, high)
2      if (low < high)
3          pi = partition(Arr, low, high)
4          quickSort(Arr, low, pi - 1)
5          quickSort(Arr, pi + 1, high)

```



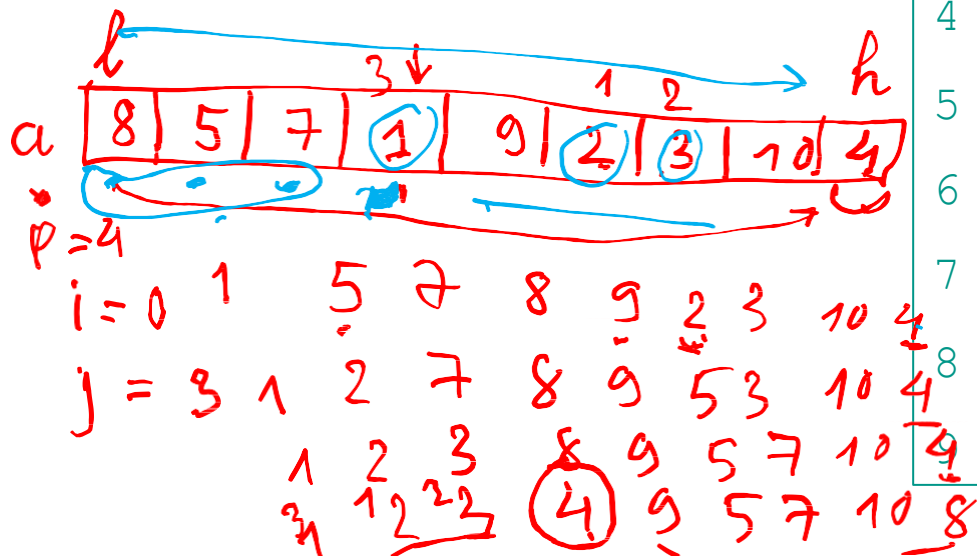
// same pointer
 $a[pivot]$

$l = 0, h = 5;$
 8 2 1 5 9 (3)
 (2) 1 3 (5 9 8)
 1 2 3 5 8 9 1

Partition Lomuto

1 2 3 3 1 2 4 ,

Duyệt từ trái qua phải và dồn các số nhỏ hơn pivot về bên trái cho đến khi mọi phần tử đều được duyệt
Phần bên trái sẽ các số nhỏ hơn pivot

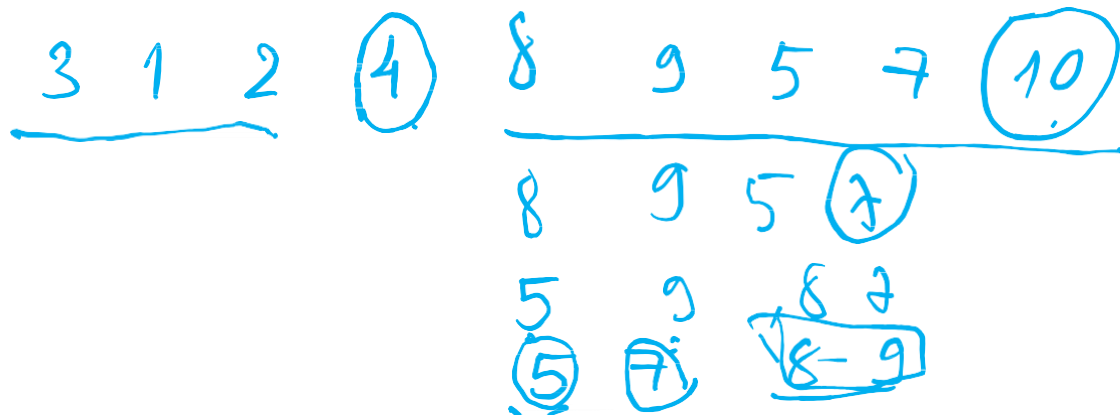


```
1 lomutoPartition(Arr, low, high)
2 pivot = Arr[high]
3 i = low - 1
4 for (j = low; j < high; j++)
5     if (Arr[j] < pivot)
6         i++
7         swap(Arr[i], Arr[j])
8 swap(Arr[i + 1], Arr[high])
9 return i + 1
```

Quick Sort

8 5 7 3 9 1 2 10 4

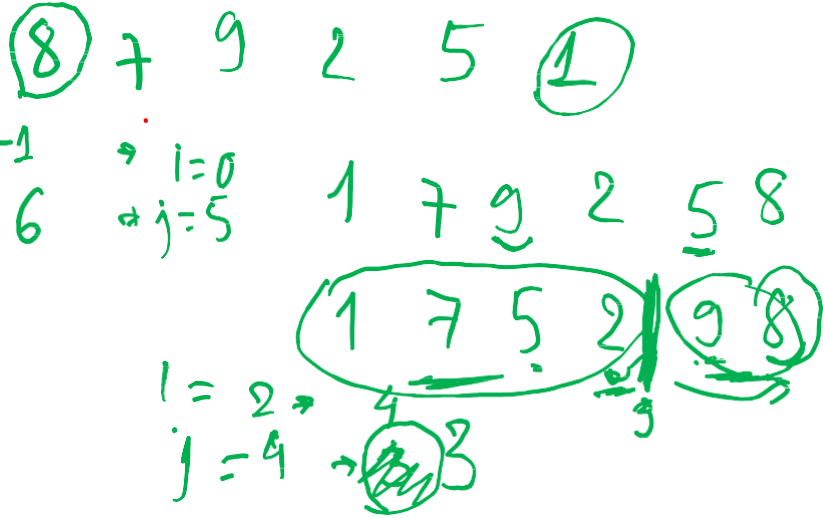
```
1 quickSort(Arr, low, high)
2   if (low < high)
3       pi = lomutoPartition(Arr, low, high)
4       quickSort(Arr, low, pi - 1)
5       quickSort(Arr, pi + 1, high)
```



Partition Hoare

u v
 $u > pivot$ $v < pivot$

Đổi chỗ hai số $u > pivot$, $v < pivot$ nếu u đứng trước v cho đến khi không còn cặp nào như vậy



```
1 hoarePartition(Arr, low, high)
2     pivot = Arr[low]
3     i = low - 1
4     j = high + 1
5     while (true)
6         do
7             i++
8             while (Arr[i] < pivot)
9         do
10            j--
11            while (Arr[j] > pivot)
12        if (i >= j)
13            return j
14        swap(Arr[i], Arr[j])
```

Quick Sort

8 7 9 2 5 1

```
1 quickSort(Arr, low, high)
2     if (low < high)
3         pi = hoarePartition(Arr, low, high)
4         quickSort(Arr, low, pi)
5         quickSort(Arr, pi + 1, high)
```

1 || 7 _ 5 2

Quick Sort with Random Pivot

```
1 QuickSort(Arr, L, H)
```

```
2   if (L < H)
```

```
3       k = Arr[random(L, H)];
```

```
4       i = L; j = H;
```

```
5       do{
```

```
6           while (A[i] < k) i++
```

```
7           while (A[j] > k) j--
```

```
8           if (i <= j)
```

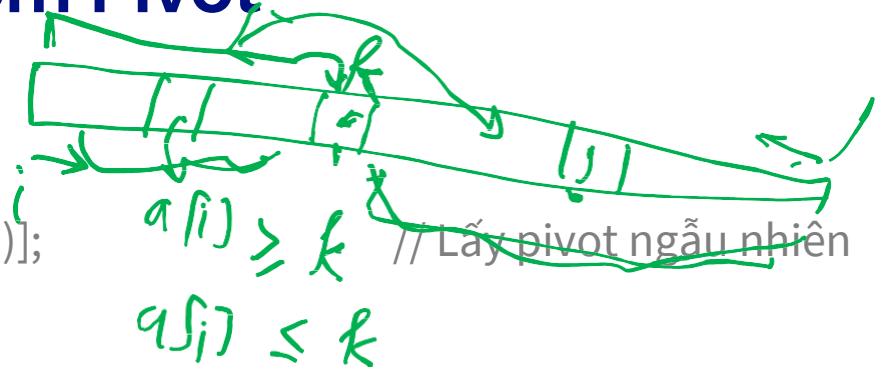
```
9               if (i < j) Swap(A[i], A[j])
```

```
10              ++i; --j
```

```
11          while (i < j)
```

```
12              QuickSort(L, j)
```

```
13              QuickSort(i, H)
```



Quick Sort

Độ phức tạp

Trường hợp tốt nhất: $O(n \log n)$

Trường hợp xấu nhất: $O(n^2)$

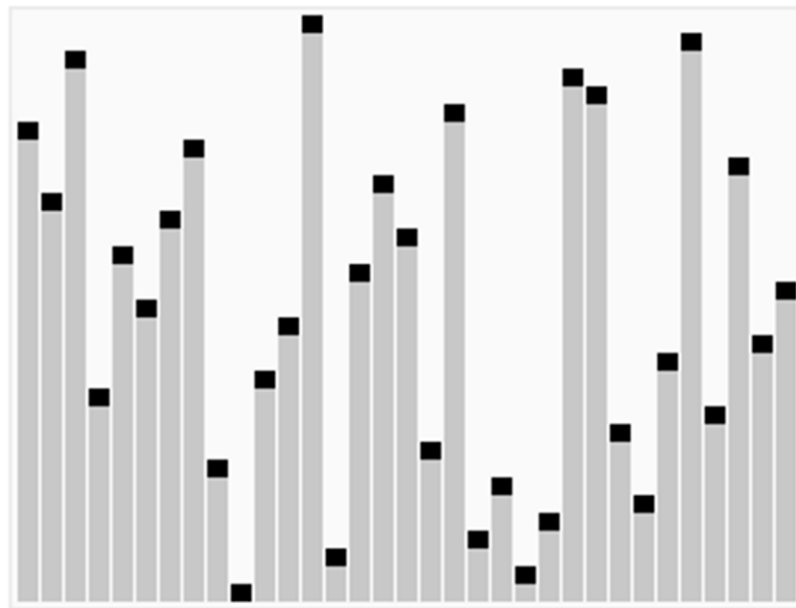
Không giữ nguyên thứ tự của các phần tử bằng nhau

Là thuật toán có vận tốc trung bình là nhanh nhất

Heap Sort

Heap Sort

- Là một thuật toán sắp xếp dựa trên so sánh, nổi bật với hiệu suất ổn định.
- **Ý tưởng cốt lõi:** Thuật toán sử dụng một cấu trúc dữ liệu đặc biệt gọi là **Heap (Đống)** để quản lý dữ liệu.
- Nó biến mảng đầu vào thành một **Max-Heap** (đống cực đại), trong đó phần tử lớn nhất luôn nằm ở gốc.
- Sau đó, nó lần lượt lấy phần tử lớn nhất ra và xây dựng mảng đã sắp xếp.



Cấu trúc Heap

Heap là một cấu trúc dữ liệu dạng cây nhị phân gần hoàn chỉnh, thỏa mãn tính chất Heap:

- Max-Heap: Giá trị của mỗi nút cha lớn hơn hoặc bằng giá trị của các nút con.
- Min-Heap (Đồng cực tiểu): Ngược lại, giá trị nút cha nhỏ hơn hoặc bằng giá trị các nút con.

Mảng A có thể coi là một cây nhị phân với công thức

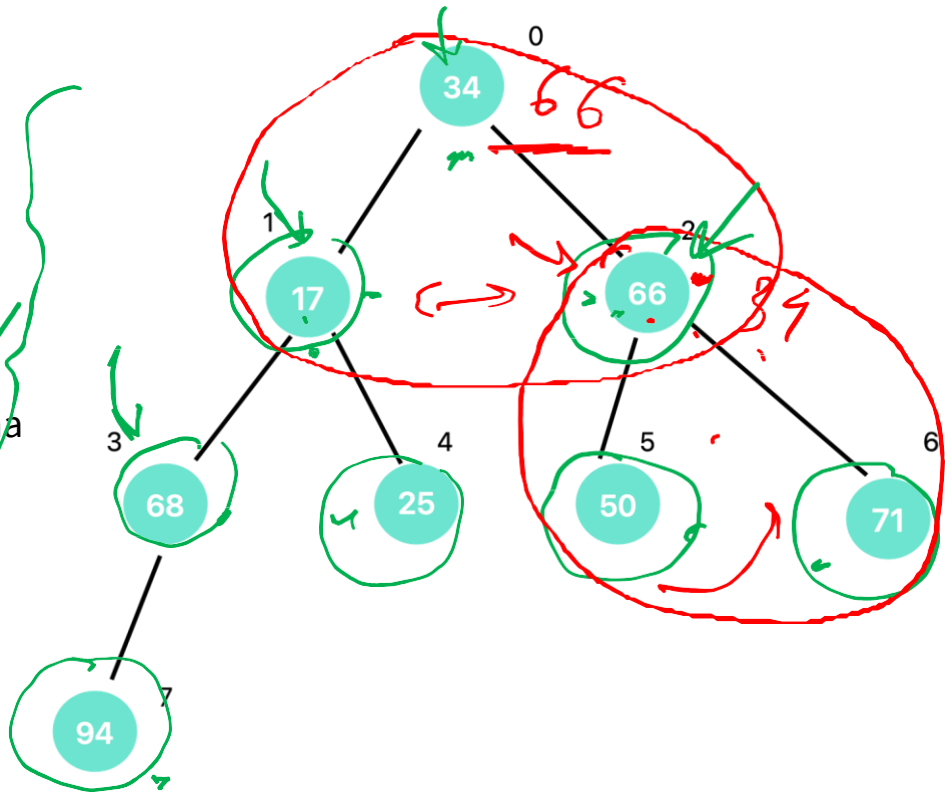
$$i\text{LeftChild}(i) = 2 * i + 1$$

$$i\text{RightChild}(i) = 2 * i + 2$$

$$i\text{Parent}(i) = \text{floor}((i - 1) / 2)$$

$$i \left[\frac{(i-1)}{2} \right]$$

0	1	2	3	4	5	6	7
34	17	66	68	25	50	71	94



Các bước của Heap Sort

Giai đoạn 1: Xây dựng Max-Heap (Build Max-Heap)

- **Mục tiêu:** Sắp xếp lại mảng ban đầu để nó tuân theo tính chất của một Max-Heap.
- **Cách làm:** Bắt đầu từ nút không phải lá cuối cùng, ta thực hiện thao tác **Heapify** (vun đống) ngược lên đến gốc. Thao tác này đảm bảo mỗi cây con đều là một Max-Heap.

Giai đoạn 2: Vòng lặp Sắp xếp

- **Mục tiêu:** Lấy từng phần tử lớn nhất ra khỏi heap và đặt vào đúng vị trí cuối mảng.
- **Cách làm (lặp lại $n-1$ lần):**
 - **Hoán đổi** phần tử gốc (lớn nhất) với phần tử cuối cùng của heap.
 - **Giảm kích thước** của heap đi 1 (loại bỏ phần tử cuối cùng đã được sắp xếp).
 - Thực hiện **Heapify** lại trên gốc để khôi phục tính chất Max-Heap cho heap đã giảm kích thước.

Thủ tục Heapify

Heapify (vun đống) là thao tác cốt lõi để duy trì hoặc sửa chữa **tính chất Heap** trong một cấu trúc dữ liệu dạng cây. Nó không phải là một thuật toán sắp xếp hoàn chỉnh mà là một thủ tục nền tảng cho **Heap Sort**.

Mục tiêu của heapify là nhận vào một nút trong cây mà có thể đang vi phạm tính chất Heap và điều chỉnh cây con bắt nguồn từ nút đó để nó trở thành một Heap hợp lệ.

Điều kiện tiên quyết quan trọng: Thuật toán heapify giả định rằng **tất cả các cây con bên dưới nút đang xét đã là Heap hợp lệ**.

Hoạt động của Heapify

Các bước thực hiện trên một nút i:

- **Tìm phần tử lớn nhất:** So sánh giá trị của nút i với giá trị của hai nút con của nó (con trái và con phải). Xác định phần tử lớn nhất trong ba phần tử này.
- **Kiểm tra và Hoán đổi:**
 - Nếu phần tử lớn nhất chính là nút i (nút cha), thì cây con này đã thỏa mãn tính chất Max-Heap. Quá trình kết thúc.
 - Nếu phần tử lớn nhất là một trong hai nút con, hãy **hoán đổi** giá trị của nút i với nút con lớn nhất đó.
- **Đệ quy:** Việc hoán đổi có thể làm mất tính chất Heap của cây con ở phía dưới. Do đó, hãy gọi đệ quy heapify trên nút con vừa được hoán đổi để tiếp tục "sàng" phần tử xuống cho đến khi nó tìm được vị trí đúng.

Minh họa hoạt động của Heapify

```
void heapify(std::vector<int>& arr, int n, int i) {
```

```
    int largest = i; // Khởi tạo nút lớn nhất là nút gốc
```

```
    int left = 2 * i + 1; // Chỉ số của con trái
```

```
    int right = 2 * i + 2; // Chỉ số của con phải
```

// C++ / C

```
    // Nếu con trái tồn tại và lớn hơn gốc
```

```
    if (left < n && arr[left] > arr[largest]) largest = left;
```

```
    // Nếu con phải tồn tại và lớn hơn phần tử lớn nhất hiện tại
```

```
    if (right < n && arr[right] > arr[largest]) largest = right;
```

largest =

```
    // Nếu nút lớn nhất không còn là gốc, ta cần hoán đổi
```

```
    if (largest != i) {
```

```
        std::swap(arr[i], arr[largest]);
```

66 lên
31 xuống

```
    // Sau khi hoán đổi, cây con bị ảnh hưởng có thể mất tính chất Max-Heap,
```

```
    // vì vậy ta cần đệ quy vun đống lại cây con đó.
```

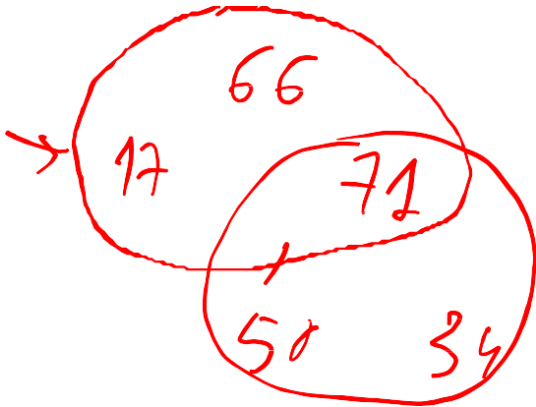
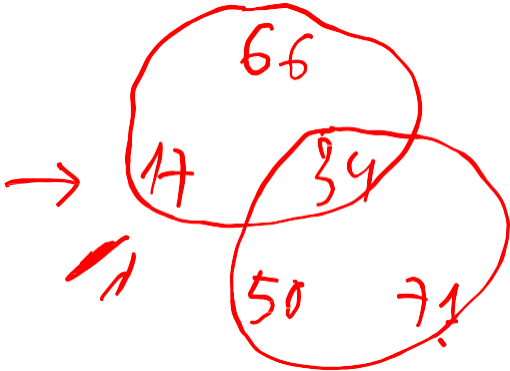
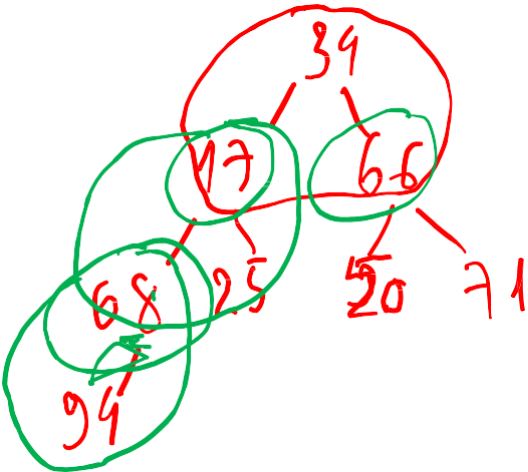
```
    heapify(arr, n, largest);
```

```
    }
```

Minh họa hoạt động của Heapify

4, 10, 8, 5, 1

34 17 66 68 25 50 71 99



HeapSort

```
void heapSort(std::vector<int>& arr) {  
    int n = arr.size();
```

```
    // ---- Giai đoạn 1: Xây dựng Max-Heap từ mảng ban đầu ----
```

```
    // Ta bắt đầu từ nút không phải lá cuối cùng ( $n / 2 - 1$ ) và đi ngược lên gốc.
```

```
    for (int i = n / 2 - 1; i >= 0; i--) {  
        heapify(arr, n, i);  
    }
```

why?

max-heap

```
    // ---- Giai đoạn 2: Lần lượt trích xuất phần tử từ heap ----
```

```
    // Vòng lặp này sẽ đặt phần tử lớn nhất về cuối mảng và vun đóng lại phần còn lại.
```

```
    for (int i = n - 1; i > 0; i--) {
```

```
        // Di chuyển gốc hiện tại (phần tử lớn nhất) về cuối mảng
```

```
        std::swap(arr[0], arr[i]);
```

```
        // Gọi heapify trên heap đã giảm kích thước (kích thước là i)
```

```
        // để đưa phần tử lớn nhất mới lên gốc.
```

```
        heapify(arr, i, 0);
```

```
    }
```

```
}
```

17



Heap Sort

Độ phức tạp tạo cây Heap Max $O((n/2)\log n)$

Độ phức tạp phần sắp xếp $O(n\log n)$ 

Vận tốc có tính ổn định không liên quan đến INPUT 

Khi tìm một vài số lớn nhất thì không cần sắp xếp toàn bộ (giống Selection sort)

Không giữ nguyên thứ tự của các phần tử bằng nhau

Counting Sort

Counting Sort

Đếm số lần xuất hiện của mỗi phần tử

Là thuật toán sắp xếp không so sánh

Input Data

0	4	2	2	0	0	1	1	0	1	0	2	4	2
---	---	---	---	---	---	---	---	---	---	---	---	---	---

Count Array

0	1	2	3	4
5	3	4	0	2

Sorted Data

0	0	0	0	0	1	1	1	2	2	2	2	4	4
---	---	---	---	---	---	---	---	---	---	---	---	---	---

Counting Sort

- Đếm số lần xuất hiện của các phần tử trong danh sách
- In lại danh sách theo số đã đếm được

Input Data

0	4	2	2	0	0	1	1	0	1	0	2	4	2
---	---	---	---	---	---	---	---	---	---	---	---	---	---

Count Array

0	1	2	3	4
5	3	4	0	2

Sorted Data

0	0	0	0	0	1	1	1	2	2	2	2	4	4
---	---	---	---	---	---	---	---	---	---	---	---	---	---

Counting Sort

```
1 countingSort(input, k)
2   for (i = 0; i < length(input); i++)
3       count[input[i]]++
4   index = 0
5   for (i = 0; i <= k; i++)
6       for (j = 0; j < count[i];
7           j++)
8           output[index] = i
9           index++
9   return output
```

Handwritten notes:
-5 → 0 → 15
10

Input Data

0	4	2	2	0	0	1	1	0	1	0	2	4	2
---	---	---	---	---	---	---	---	---	---	---	---	---	---

Count Array

0	1	2	3	4
5	3	4	0	2

Sorted Data

0	0	0	0	0	1	1	1	2	2	2	2	4	4
---	---	---	---	---	---	---	---	---	---	---	---	---	---

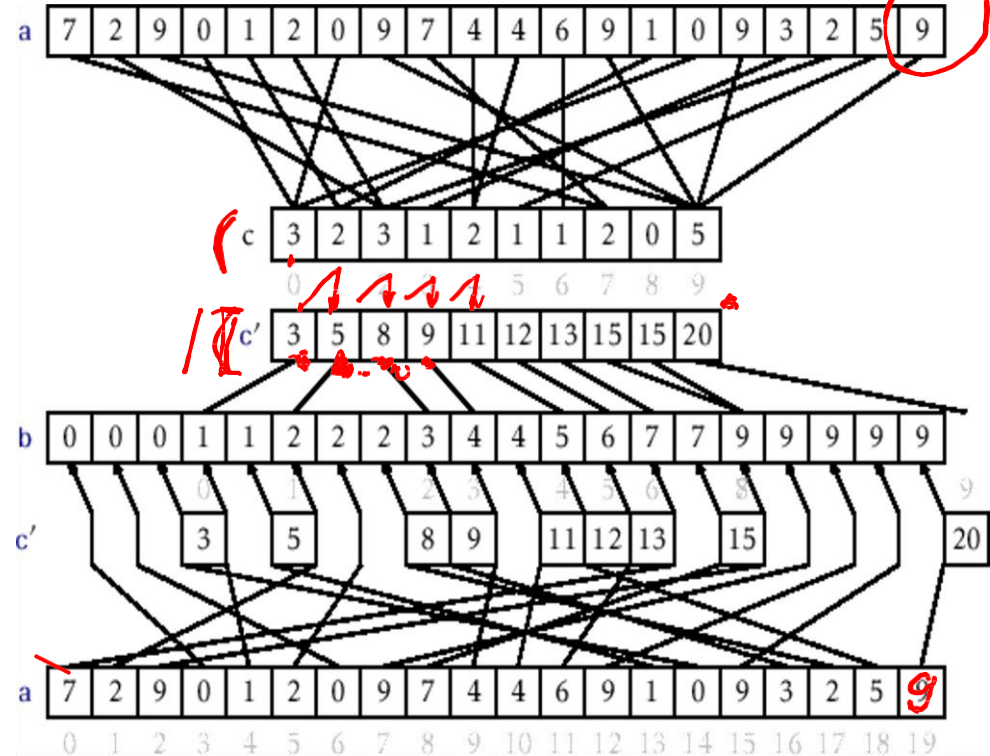
Counting Sort

for (i = 0 → k)
for (j = 0 → c[i]) print(i, " ");

```

1 countingSort(A, k)
2   for (i = 0; i < length(A); i++)
3     C[A[i]]++
4   for (i = 1; i <= k; i++)
5     C[i] = C[i] + C[i - 1]
6   for (i = length(A) - 1; i >= 0; i--)
7     j = input[i]
8     C[j] --
9     B[C[j]] = j
10  return B
    
```

$O(n + k)$



Counting Sort

Độ phức tạp là $O(n+k)$, với k là khoảng từ min đến max giá trị

Không có độ ổn định

Phù hợp với dữ liệu có k nhỏ

Contact

TS. Nguyễn Thanh Bình

binh.nguyenthanh@phenikaa-uni.edu.vn

